

Prediccion de Abandono de Clientes (Churn)

AndesLink Servicios Digitales S.A.

Alumnos: Facundo Zubeldia · Gonzalo Martin Gonzalez Nastovich · Agustin Meza
Profesor: Ing. Diego Mosquera
Fecha: Junio 2026

1. Objetivo de la Entrega

En esta segunda entrega se expone el modelo entrenado en la Entrega 1 como un servicio productivo. El objetivo es convertir el modelo serializado (model_churn.pkl) en una API REST consumible por sistemas externos, complementada por una interfaz grafica para usuarios no tecnicos, todo empaquetado en contenedores Docker para garantizar reproducibilidad del despliegue.

2. Arquitectura Local del Sistema

La solucion se compone de dos servicios independientes orquestados con Docker Compose, que se comunican a traves de la red interna de Docker:

Servicio	Tecnologia	Puerto	Funcion
API de inferencia	FastAPI + Uvicorn	8000	Recibe datos JSON, devuelve prediccion
Dashboard GUI	Streamlit	8501	Interfaz visual para usuarios finales

Flujo de inferencia:

- El usuario ingresa datos del cliente en el dashboard Streamlit.
- Streamlit envia una solicitud POST a la API FastAPI con los datos en formato JSON.
- La API carga el pipeline serializado (preprocesador + modelo), aplica las transformaciones y genera la prediccion.
- La API responde con: probabilidad de churn, prediccion binaria (0/1) y nivel de riesgo (bajo/medio/alto).

5. El dashboard muestra el resultado de forma visual con metricas, indicador de riesgo y resumen del cliente.

Comunicacion entre contenedores:

Dentro de Docker Compose, el dashboard llama a la API usando el nombre del servicio como hostname (`http://api:8000`) en lugar de localhost. Esto es posible gracias a la red interna que Docker Compose crea automaticamente entre los contenedores. La variable de entorno `API_URL` permite configurar este comportamiento sin modificar el codigo.

3. API de Inferencia (FastAPI)

3.1 Descripción del endpoint

La API expone dos endpoints:

- GET /: health check — verifica que el servicio esta activo.
- POST /predict: endpoint principal — recibe datos de un cliente y devuelve la prediccion.

Contrato de entrada (ClienteInput):

15 campos obligatorios con validacion de tipos y rangos: tenure_months (int, >= 0), monthly_charge (float, >= 0), contract_type (str), internet_service (str), has_streaming (int, 0 o 1), entre otros. FastAPI valida automaticamente cada campo usando Pydantic y devuelve error 422 si algun campo es invalido o falta.

Contrato de salida (PrediccionOutput):

- churn: 0 o 1 (prediccion binaria)
- probabilidad_churn: float entre 0.0 y 1.0
- riesgo: 'bajo' (< 0.4), 'medio' (0.4-0.7) o 'alto' (>= 0.7)

3.2 Documentacion interactiva (Swagger UI)

FastAPI genera automaticamente documentacion interactiva en /docs donde se pueden probar los endpoints sin herramientas adicionales:

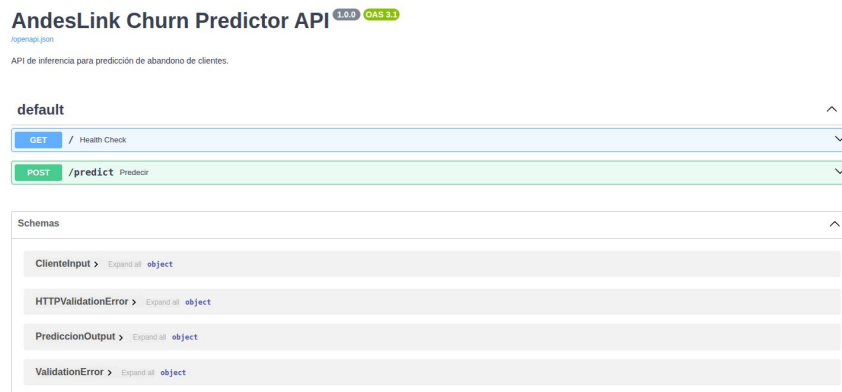


Figura 1: Documentacion Swagger de la API con endpoints GET / y POST /predict.

4. Dashboard de Prediccion (Streamlit)

El dashboard permite a usuarios no tecnicos de AndesLink ingresar datos de un cliente y obtener la probabilidad de churn de forma inmediata. Consume la API FastAPI internamente.

Funcionalidades:

- Formulario lateral con sliders y selectores para los 15 campos del modelo.
- Boton de prediccion que invoca el endpoint POST /predict.
- Metricas: probabilidad de churn, prediccion y nivel de riesgo.
- Indicador visual de riesgo con zonas Bajo/Medio/Alto.
- Tabla resumen del perfil del cliente analizado.
- Manejo de errores: muestra mensaje claro si la API no esta disponible.

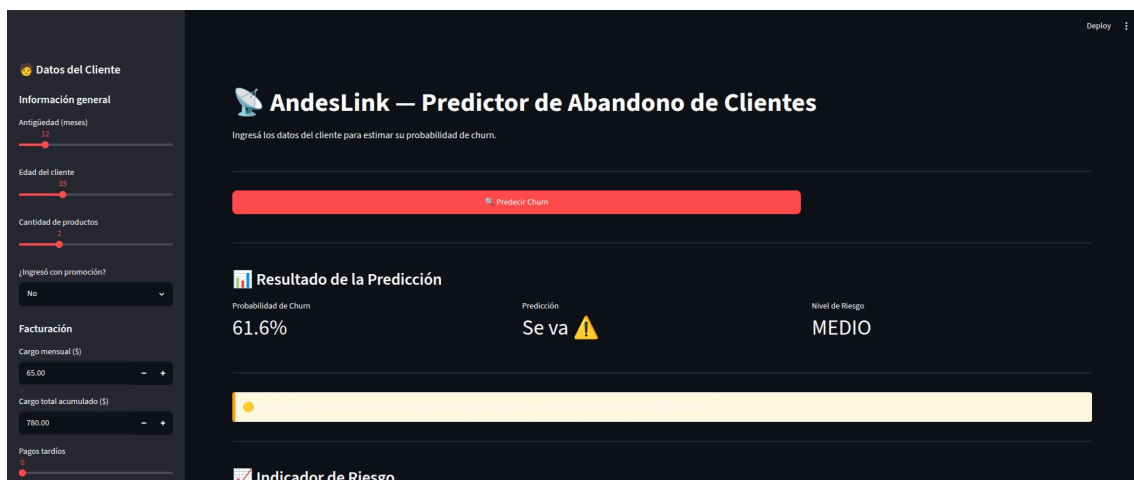


Figura 2: Dashboard mostrando prediccion de churn con probabilidad 61.6%, riesgo MEDIO.



Figura 3: Indicador de riesgo y resumen del cliente analizado.

5. Contenedorizacion con Docker

Ambos servicios estan empaquetados en imagenes Docker independientes, orquestadas con Docker Compose. Esto garantiza que la solucion corre de manera identica en cualquier maquina con Docker instalado, sin depender de configuraciones locales.

Archivos de contenedorizacion:

- Dockerfile: imagen para la API FastAPI basada en python:3.11-slim.
- Dockerfile.streamlit: imagen para el dashboard Streamlit.
- docker-compose.yml: orquesta ambos servicios, define puertos, dependencias y red interna.
- requirements-docker.txt: dependencias minimas para produccion (sin MLflow, DVC ni herramientas de desarrollo).

Comandos de operacion:

Levantar los servicios:

```
docker-compose up --build -d
```

Verificar estado:

```
docker-compose ps
```

Detener servicios:

```
docker-compose down
```

El despliegue fue validado de forma independiente por los tres integrantes del equipo, clonando el repositorio en distintas maquinas y ejecutando docker-compose up --build -d sin necesidad de configuracion adicional, confirmando la portabilidad de la solucion.

6. Tests Automatizados (pytest)

Se implementaron 7 tests automatizados con pytest que cubren los casos criticos de la API. Todos los tests corren en menos de 2 segundos usando TestClient de FastAPI sin necesidad de levantar el servidor. Ademas, los tests corren automaticamente en GitHub Actions con cada push al repositorio, junto con un segundo workflow que valida que ambas imagenes Docker se construyen correctamente.

Test	Descripcion	Resultado
test_health_check	GET / devuelve 200 con status ok	PASSED
test_prediccion_cliente_valido	POST /predict con datos validos devuelve 200	PASSED
test_prediccion_payload_incompleto	Payload sin campos devuelve 422	PASSED
test_prediccion_tipo_incorrecto	Tipos incorrectos devuelven 422	PASSED
test_prediccion_cliente_bajo_riesgo	Cliente fiel tiene probabilidad < 0.7	PASSED
test_prediccion_cliente_alto_riesgo	Cliente en riesgo tiene probabilidad >= 0.4	PASSED
test_formato_respuesta	Respuesta tiene exactamente los campos esperados	PASSED

Tabla 1: Resultados de los 7 tests automatizados. El reporte completo esta en [reports/test_report.html](#).

7. Conclusiones y Proximos Pasos

Se completo exitosamente la Entrega 2 con una solucion de despliegue local completamente funcional. El modelo de Regresion Logistica entrenado en la Entrega 1 ahora es accesible como servicio REST y cuenta con una interfaz grafica operativa.

Mejoras incorporadas respecto a la Entrega 1:

- Split estratificado (stratify=y): garantiza que el desbalance 66/34 se mantiene en train y test.
- Recall mejorado de 0.704 a 0.729 gracias al split estratificado.
- Dataset incluido directamente en el repositorio, resolviendo la limitacion de portabilidad de DVC senalada en la Entrega 1.

Limitaciones actuales:

- Sin autenticacion en la API; en produccion se implementaria OAuth2 o API keys.
- Sin logging estructurado de predicciones; necesario para auditoria en produccion.
- El dataset se incluye directamente en el repositorio (no solo via dvc pull) dado que el remote configurado es local. Para un entorno multi-usuario se recomendaria un remote en la nube (S3, GCS) con autenticacion por Service Account.

Proximos pasos (Entrega 3):

- Instrumentar metricas tecnicas con Prometheus (requests, latencia, errores).
- Visualizar metricas operacionales en Grafana.
- Implementar monitoreo de drift con Evidently.
- Generar reportes de calidad del modelo en produccion.